NOTARDEX

Remaining Bugs, Gaps & Issues Audit

Comprehensive Analysis • April 2026 • Version 2

Previously Unreported Findings Only

Executive Summary

This audit identifies 78 NEW bugs, gaps, and issues not covered in any previous audit documents. These findings span security vulnerabilities, data integrity bugs, UX problems, accessibility gaps, and compliance blind spots across the client portal, admin panel, booking flow, RON session, and edge functions.

Severity	Count	Impact
CRITICAL	9	Security vulnerabilities, data loss, compliance violations
HIGH	22	Broken features, data corruption, user-facing errors
MEDIUM	31	UX degradation, missing validation, incomplete features
LOW	16	Code quality, minor UX issues, optimization opportunities

1. Critical Security Issues (9)

1.1 Client-Side Account Deletion Without Transaction

File: ClientPortal.tsx line 826

The account deletion flow executes 7 sequential DELETE queries client-side without a database transaction. If any query fails mid-way, the account is left in an inconsistent state with orphaned records. Additionally, auth.users is not deleted (only profile and roles), so the user cannot re-register with the same email.

> ⚠ *CRITICAL: Move to a server-side delete-account edge function with a single transaction. Include auth.admin.deleteUser() call.*

1.2 Cancel Reason Silently Discarded

File: ClientPortal.tsx line 197

The cancel appointment function captures cancelReason in state (line 784) but NEVER includes it in the update query. The reason is silently discarded, making it impossible to track why appointments were cancelled.

> ⚠ *Add cancel_reason to the update payload: .update({ status: 'cancelled', cancel_reason: cancelReason })*

1.3 Edge Functions Without Authentication

Files: extract-email-leads/index.ts, hubspot-sync/index.ts

Both functions use SUPABASE_SERVICE_ROLE_KEY directly without verifying the caller's JWT. Any anonymous request can trigger lead extraction or HubSpot sync operations, potentially exposing customer data.

> ⚠ *Add JWT verification: const authHeader = req.headers.get('Authorization'); verify token before processing.*

1.4 Client-Side Payment Status Mutation

File: ClientPortal.tsx line 667

Payment status is updated client-side: supabase.from('payments').update({ status: 'paid' }). A user could intercept this call and mark any payment as paid without actually processing payment through Stripe.

> ⚠ *Payment status should ONLY be updated by the stripe-webhook edge function upon receiving a confirmed payment_intent.succeeded event.*

1.5 Profile Update Lacks Input Validation

File: ClientPortal.tsx line 206

The saveProfile function sends form values directly to the database without sanitizing or validating: full_name can contain HTML/script, phone has no format validation server-side, and zip code has no numeric check. While RLS limits row access, XSS via stored profile data is possible when admin views client profiles.

1.6 Chat Messages Not Sanitized

File: ClientPortal.tsx line 228

Chat messages are inserted raw (chatInput.trim()) without DOMPurify sanitization. If messages are rendered with dangerouslySetInnerHTML elsewhere (admin chat panel), this creates a stored XSS vector.

1.7 Notification Bell Non-Functional

File: ClientPortal.tsx line 285-289

The notification bell icon shows a badge dot but clicking it does nothing. There is no dropdown, no notification list, and no onClick handler. The badge calculation only counts pending payments and unread chats, missing appointment status changes, document approvals, and correspondence.

1.8 Remember Me Checkbox Non-Functional

File: Login.tsx line 160

The 'Remember me' checkbox state (rememberMe) is captured but never used. Supabase auth always persists to localStorage regardless. The checkbox creates a false sense of control.

1.9 Search Bar Non-Functional

File: ClientPortal.tsx line 283

The portal search bar is a plain HTML input with no onChange handler, no state binding, and no search logic. It renders as a static decoration.

2. Data Integrity Bugs (14)

2.1 AdminOverview Revenue From Wrong Source

File: AdminOverview.tsx line 68

Revenue is calculated from notary_journal.fees_charged instead of payments table. This shows anticipated fees, not actual collected revenue. Refunds, unpaid invoices, and partial payments are not reflected.

2.2 660+ Type-Unsafe Casts Across 44 Files

The codebase contains 660+ 'as any' type casts across 44 files. These mask TypeScript errors and hide potential runtime failures where the database schema has diverged from the code's assumptions.

2.3 Reviews Table vs client_feedback Table Confusion

ClientPortal.tsx writes to 'reviews' table (line 683) but there also exists 'client_feedback' and 'service_reviews' tables. Three separate feedback storage locations create data fragmentation.

2.4 Booking Draft Collision

File: BookAppointment.tsx line 145-150

Draft auto-save uses .select('id').eq('user_id', user.id).limit(1).single() which fails if the user has multiple abandoned drafts. The .single() call throws when no row exists (204 error), and there's no cleanup for stale drafts.

2.5 Double-Booking Race Condition

File: BookAppointment.tsx line 464-466

The day capacity check (max_appointments_per_day) uses a client-side count query followed by insert. Two simultaneous bookings can both pass the check and exceed the limit. The prevent_double_booking trigger only checks exact time matches, not daily limits.

2.6 Realtime Channel Leak on Portal Tab Switch

File: ClientPortal.tsx line 160-180

Four realtime channels are created but the cleanup function (line 182) only removes them on component unmount. The chat message filter uses .or() with string interpolation, which could fail silently if user.id contains special characters.

2.7 Stale Closure in Auth State Change

File: AuthContext.tsx line 102-116

The initialSessionHandled flag pattern uses a mutable variable across an async boundary. If getSession resolves after a rapid login/logout/login cycle, the flag may never reach true for the second session, causing role fetch to be skipped.

2.8 Tool Generation Count Never Incremented

File: ToolRunner.tsx line 70-75

The usage count reads from tool_generations table but there's no corresponding INSERT to increment the count after a successful generation. The free tier limit (2 generations) is checked but never enforced since the count stays at 0.

2.9 E-Seal Hash Uses Non-Deterministic Input

File: RonSession.tsx line 665

The SHA-256 hash includes new Date().toISOString() which means the same document produces different hashes each time. This makes independent verification impossible. The hash should be computed from the actual document content, not metadata.

2.10 Appointment Date Validation Gap

File: BookAppointment.tsx line 522-524

Date validation compares string dates (date < today) which works for ISO format but the today variable uses toISOString().split('T')[0] which is UTC-based. A user booking at 11 PM EST would see 'today' as tomorrow's date in UTC, allowing past-date bookings.

2.11 Orphaned Documents on Appointment Deletion

File: ClientPortal.tsx line 826

Account deletion deletes documents from the database but does NOT delete the actual files from Supabase storage. Orphaned files remain in the 'documents' bucket indefinitely, consuming storage and creating a data retention compliance issue.

2.12 Missing Pagination on Client Portal Data Fetch

File: ClientPortal.tsx line 124-134

All client data (appointments, documents, payments, reviews, services, correspondence, apostille, reminders) is fetched without pagination limits. A power user with hundreds of documents will hit the Supabase 1000-row default limit silently, showing incomplete data.

2.13 Journal Entry Created Without KBA Verification Data

File: RonSession.tsx line 615-640

The notary_journal insert includes credential_analysis as a JSON field but the KBA data (kbaAttempts, kba score) comes from local state that resets on page refresh. If the notary refreshes during Phase 3-4, KBA data is lost.

2.14 Session Timeout Warning Fires Prematurely

File: AuthContext.tsx line 66-69

The session expiry timer uses session.expires_at * 1000 for milliseconds but doesn't account for token auto-refresh. When autoRefreshToken succeeds, the old timers still fire, showing false timeout warnings.

3. User Experience Issues (18)

3.1 Portal Tab Overflow on Mobile

File: ClientPortal.tsx line 326-341

14 tab triggers are rendered in a single horizontal scrollable row. On mobile devices, tabs are cut off and the scrollbar is hidden (scrollbar-hide). Users may never discover tabs beyond the visible viewport (typically only 4-5 visible on a 375px screen).

3.2 No Loading State for Apostille/Correspondence/Services Tabs

File: ClientPortal.tsx

The initial data fetch loads all 10 queries in parallel (line 124). If any single query is slow, the entire portal shows PortalLoadingSkeleton. Individual tabs have no independent loading states, so switching to a fast tab while a slow query runs blocks everything.

3.3 Chat Auto-Scroll Triggers on Any Message Change

File: ClientPortal.tsx line 234

The useEffect that scrolls to chatEndRef fires on any change to chatMessages, including initial load and read status updates. This causes jarring scroll jumps when the user is reading earlier messages.

3.4 Empty State Missing for Leads Tab

File: ClientPortal.tsx

The Leads tab (referenced in TabsTrigger) has no corresponding TabsContent. Clicking 'Leads' shows a blank panel with no empty state or explanation.

3.5 QR Code URL Not User-Specific

File: ClientPortal.tsx line 259

The QR code URL is just window.location.origin + '/mobile-upload' with no user token. Anyone scanning the QR code lands on a generic upload page. Files uploaded may not be linked to the correct user if the scanner is not logged in.

3.6 Booking Wizard No Back Button on Step 1

File: BookAppointment.tsx

Step 1 has no way to return to the services page. The browser back button works but there's no visible back navigation, making users feel trapped.

3.7 RON Session No Auto-Save

File: RonSession.tsx

The RON session is 2,118 lines with extensive state but no auto-save. If the browser crashes or the user accidentally closes the tab during a session, all progress (notes, ID verification, KBA status) is lost. saveSessionData() must be explicitly clicked.

3.8 Date Display Timezone Inconsistency

Files: ClientPortal.tsx line 118, utils.ts

ClientPortal uses a local formatDate with 'T00:00:00' appended to prevent timezone shift. Other pages use the shared formatDate from utils.ts which may not have this safeguard, causing dates to display as the previous day in western US timezones.

3.9 No Confirmation for Apostille Submission

Apostille requests submit directly without a review step or confirmation dialog. Users cannot preview their submission before it's sent.

3.10 Error Messages Not User-Friendly

Throughout the codebase, Supabase error.message is shown directly to users (e.g., 'relation "reviews" violates row level security'). These technical messages should be translated to user-friendly language.

3.11 Portal Tabs Missing: Leads

The TabsList includes a 'Leads' trigger referencing a PortalLeadsTab import file but the TabsContent for leads is never rendered in the JSX. The tab exists in navigation but shows nothing when clicked.

3.12 Document Explain Only Works for Text Files

File: ClientPortal.tsx line 239-241

The explainDocument function downloads the file and calls .text() on the blob. For PDF files, this returns raw binary as garbled text, making the AI explanation useless for the most common document format.

3.13 No Offline Indicator on Booking Page

The OfflineIndicator component exists but is not rendered on the BookAppointment page. If a user loses connection mid-booking, the submit silently fails with a generic error.

3.14 Profile Edit Modal Can Lose Data

File: ClientPortal.tsx line 806-828

The edit profile dialog has no unsaved changes warning. Clicking outside the dialog or pressing Escape discards all edits without confirmation.

3.15 Appointment Status Labels Not Consistent

Admin uses 'id_verification', 'kba_pending', 'in_session' but client portal only filters for 'scheduled', 'confirmed'. Status labels like 'kba_pending' are shown raw with replace(/_/g, ' ') creating ugly 'kba pending' text.

3.16 No Deep Link for Specific Appointment

The portal supports tab deep links (?tab=appointments) but not individual appointment deep links (?tab=appointments&id=xxx). Users clicking email notification links land on the generic appointments list.

3.17 Admin Sidebar Flat Navigation

33 sidebar items in a flat list without grouping, collapsible sections, or search. Finding a specific page requires scrolling through the entire list.

3.18 No Keyboard Shortcut Help

The CommandPalette component exists but there's no visible hint about Cmd+K to open it. No keyboard shortcut documentation or help overlay.

4. Accessibility Gaps (8)

4.1 Missing ARIA Labels on Icon-Only Buttons

Multiple icon-only buttons throughout the admin panel lack aria-label attributes (e.g., refresh buttons, filter toggles, status change buttons in AdminAppointments.tsx).

4.2 Color-Only Status Indicators

Appointment and document status badges rely solely on color to convey meaning. Users with color vision deficiency cannot distinguish between 'scheduled' (blue), 'confirmed' (green), and 'in_session' (amber).

4.3 No Focus Management After Tab Switch

Switching tabs in the client portal does not move focus to the new tab content. Screen reader users must navigate past all tabs to reach the content area.

4.4 Missing Skip Links in Admin Panel

The admin dashboard has a sidebar + main content layout but no skip link to jump past the 33-item sidebar navigation.

4.5 Star Rating Not Keyboard Accessible

File: ClientPortal.tsx line 678

Review star rating uses raw <button> elements but they have no keyboard focus indicators, no aria-label per star, and no role='radiogroup' semantics.

4.6 Tooltip Content Not Accessible

Tooltips across the app use hover-only triggers with no keyboard equivalent. Screen reader users and keyboard-only users cannot access tooltip content.

4.7 Chat Input Missing Label Association

The chat message input field has no associated <label> element. Screen readers announce it as an unlabeled text input.

4.8 Auto-Scroll Announcement Missing

New chat messages trigger auto-scroll but there's no aria-live region to announce incoming messages to screen reader users.

5. Ohio Compliance Gaps (6)

5.1 Recording URL Not Validated

File: RonSession.tsx line 562-564

The recording URL check only verifies !recordingUrl.trim() (non-empty). It doesn't validate that the URL points to an actual recording, that the recording is accessible, or that it meets the Ohio 10-year retention requirement.

5.2 No KBA Provider Integration

KBA verification in Phase 2 is tracked as a boolean toggle (kbaCompleted) set manually by the notary. There's no integration with an actual KBA provider (IDology, Socure, or LexisNexis). The notary self-reports KBA completion.

5.3 No Multi-Document Journal Entries

File: RonSession.tsx line 615

The finalization creates a single journal entry per session. Ohio ORC requires a separate journal entry per notarial act. If multiple documents are notarized in one session, only one journal entry is created.

5.4 Witness Verification Incomplete

The RON session has witness toggle state but no workflow for verifying witness identity. Ohio ORC §147.64(D) requires witnesses to be identified in the same manner as signers for certain document types.

5.5 Commission Expiry Check Only on Session Load

File: RonSession.tsx line 756

Commission expiry is checked when the page loads but not during the session. A session started at 11:58 PM on commission expiry day could be finalized at 12:05 AM the next day when the commission is technically expired.

5.6 No Signer Acknowledgment Persistence Pre-Finalize

The signer's acknowledgment (e-sign consent, recording consent) is stored in React state and only persisted to the database during finalization. If the session is interrupted, there's no proof that consent was obtained.

6. Edge Function Issues (7)

6.1 No Rate Limiting on Public Edge Functions

submit-lead and health-check are publicly accessible without rate limiting. An attacker could flood the leads table or use health-check for reconnaissance.

6.2 Email Queue No Dead Letter Queue Monitoring

process-email-queue uses pgmq with move_to_dlq but there's no monitoring or alerting when emails land in the dead letter queue. Failed emails disappear silently.

6.3 AI Tools Edge Function Allowlist Mismatch

The ai-tools edge function's TOOL_IDS allowlist is missing 'proposal-template' and 'executive-summary' which are registered in aiToolsRegistry.ts. These tools show in the UI but fail silently when run.

6.4 CORS Wildcard on All Edge Functions

All edge functions use Access-Control-Allow-Origin: '*'. This should be restricted to the actual domain (notary-prime-hub.lovable.app) for production security.

6.5 SignNow Webhook No Replay Protection

signnow-webhook doesn't verify webhook signatures or implement idempotency keys. Replayed webhook events could create duplicate records.

6.6 Stripe Webhook Missing Event Types

stripe-webhook likely handles payment_intent.succeeded but may not handle charge.refunded, payment_intent.payment_failed, or customer.subscription.* events needed for full payment lifecycle.

6.7 No Edge Function Timeout Handling

Long-running AI operations (ai-batch-process, ai-cross-document) have no timeout mechanism. Supabase edge functions have a 150-second hard limit, but operations that approach this limit return abruptly with no graceful degradation.

7. Code Quality Issues (16)

7.1 God Components

File	Lines	Issue
RonSession.tsx	2,118	Session management, UI, API calls, compliance checks all in one file
AdminAppointments.tsx	1,124	CRUD, filtering, bulk ops, email, translation, KBA in one component
BookAppointment.tsx	929	Booking wizard, pricing, validation, guest auth, slot mgmt combined
ClientPortal.tsx	840	14 tabs, profile, chat, payments, reviews, apostille, all in one
DocumentTemplates.tsx	~1000+	Template rendering, AI review, drafts, PDF generation combined

7.2 Inconsistent State Management

Admin pages use raw useState + useEffect patterns while no React Query or SWR is used for data fetching. This means no caching, no automatic re-fetching, no optimistic updates, and no deduplication of concurrent requests.

7.3 Empty Catch Blocks (60 Instances)

60 empty catch blocks across 8 files silently swallow errors, making debugging nearly impossible. Examples: JSON parse failures in streaming responses, localStorage write failures, LibreOffice rendering errors.

7.4 No Error Boundaries Per Route

A single ErrorBoundary wraps the entire app. An error in one admin page crashes the entire application including the client portal. Per-route error boundaries would isolate failures.

7.5 localStorage Used for Sensitive Draft Data

Template drafts, AI assistant chat history, brand analysis, and admin filter presets are stored in localStorage without encryption. These can contain client names, document details, and business intelligence.

7.6 No Bundle Size Optimization

All 87 pages use lazy loading (good), but large shared dependencies (recharts, tiptap, framer-motion) are not dynamically imported and load on every page regardless of whether they're used.

7.7 Missing Cleanup on Admin Pages

Most admin pages fetch data in useEffect with [] dependency but set state in async callbacks without checking if the component is still mounted. This causes 'Can't perform React state update on unmounted component' warnings during rapid navigation.

7.8 Hardcoded Admin Email in Database Function

handle_new_user() contains a hardcoded email check: IF LOWER(NEW.email) = LOWER('ShaneGoble@gmail.com'). This should reference a platform_settings key or environment variable, not a literal email address.

8. Prioritized Implementation Plan

All 78 issues organized into a 12-week remediation roadmap.

Phase 1: Critical Security Fixes (Weeks 1-2)

Issue	Priority	Effort	Action
1.1 Account deletion	P0	4h	Create delete-account edge function with transaction + auth.admin.deleteUser()
1.3 Unsecured edge functions	P0	2h	Add JWT verification to extract-email-leads and hubspot-sync
1.4 Client-side payment update	P0	3h	Remove client-side .update(), rely solely on stripe-webhook
1.6 Chat XSS	P0	1h	Add DOMPurify.sanitize() before insert and on render
1.5 Profile validation	P0	2h	Add Zod schema validation to profile update
6.4 CORS wildcard	P0	2h	Replace * with actual domain in all edge functions
6.1 Rate limiting	P1	3h	Add IP-based rate limiter to submit-lead
7.8 Hardcoded admin email	P1	1h	Move to platform_settings table lookup
1.2 Cancel reason lost	P1	30m	Add cancel_reason field to update query

Phase 2: Data Integrity (Weeks 3-4)

Issue	Priority	Effort	Action
2.1 Revenue source	P1	2h	Change AdminOverview to use payments table (status=paid) not journal fees
2.5 Double-booking race	P1	3h	Add server-side slot locking via database advisory lock or FOR UPDATE
2.8 Generation count	P1	2h	Insert into tool_generations after successful AI generation in ToolRunner
2.9 E-seal hash	P1	3h	Hash actual file content from storage, not metadata string
2.10 Timezone date bug	P1	1h	Use Intl.DateTimeFormat or dayjs for consistent local date comparison
2.11 Orphaned storage	P1	2h	Add storage.from('documents').remove() calls before DB delete
2.12 Missing pagination	P1	3h	Add .limit(50) with pagination controls on all portal queries
2.14 Session timeout	P2	2h	Clear timers when session refreshes via onAuthStateChange

Phase 3: UX & Accessibility (Weeks 5-8)

Issue	Priority	Effort	Action
3.1 Tab overflow	P1	4h	Consolidate into 4 grouped sections or add tab dropdown on mobile
1.7 Notification bell	P1	6h	Build NotificationDropdown with categorized alerts
1.9 Search bar	P1	4h	Implement fuzzy search across appointments and documents
3.7 RON auto-save	P1	4h	Add setInterval auto-save every 60 seconds during active

			session
3.4/3.11 Missing tabs	P2	3h	Wire up Leads TabsContent or remove from TabsList
3.12 PDF explain	P2	3h	Route PDFs through ocr-digitize before explain-document
4.1-4.8 Accessibility	P2	8h	ARIA labels, keyboard navigation, focus management, screen reader support
1.8 Remember me	P3	1h	Either implement session persistence toggle or remove checkbox

Phase 4: Compliance & Edge Functions (Weeks 9-10)

Issue	Priority	Effort	Action
5.2 KBA provider	P1	16h	Integrate IDology or Socure for real-time KBA verification
5.3 Multi-doc journal	P1	4h	Create journal entry per document, not per session
5.1 Recording validation	P2	4h	Add URL format + accessibility check for recording URLs
5.5 Commission check	P2	2h	Add real-time commission expiry check before finalization
5.6 Consent persistence	P2	3h	Persist consent to DB immediately when granted, not at finalization
6.3 Tool allowlist	P1	30m	Add missing tool IDs to ai-tools edge function TOOL_IDS
6.5 Webhook replay	P2	3h	Add signature verification and idempotency to signnow-webhook
6.6 Stripe events	P2	4h	Handle refund, failure, and subscription webhook events

Phase 5: Code Quality (Weeks 11-12)

Issue	Priority	Effort	Action
7.1 God components	P2	16h	Extract into hooks (useRONSession, useBooking, usePortalData) and sub-components
7.2 State management	P2	12h	Migrate admin data fetching to React Query for caching and deduplication
7.3 Empty catches	P2	4h	Add proper error logging or Sentry reporting to all 60 catch blocks
7.4 Error boundaries	P2	3h	Add ErrorBoundary per lazy-loaded route in App.tsx
2.2 Type safety	P3	20h	Replace 660 'as any' casts with proper TypeScript types
7.5 Sensitive localStorage	P3	4h	Encrypt sensitive draft data or move to sessionStorage
7.6 Bundle optimization	P3	6h	Dynamic import recharts, tiptap, framer-motion per-route

9. Conclusion

This audit identified 78 previously unreported issues: 9 critical security vulnerabilities, 14 data integrity bugs, 18 UX problems, 8 accessibility gaps, 6 compliance blind spots, 7 edge function issues, and 16 code quality concerns.

The most urgent items are the client-side payment mutation (1.4), account deletion without transaction (1.1), and unauthenticated edge functions (1.3), all of which represent active security risks in a production environment handling sensitive legal documents.

The 12-week remediation plan prioritizes security and compliance fixes first, followed by UX improvements and code quality refactoring.